

ControlNet の制御が効かなくなる条件と、その機構

映像メディア学 レポート課題

新井 滉明 (Komei Arai) / 学籍番号 48266454

東京大学大学院 情報理工学系研究科 電子情報学専攻 修士1年 / 伊庭研究室

研究テーマ: 進化計算・学習における資源の会計

ソースコード: <https://github.com/komeiall14/controlnet-failure-analysis>

本レポートで示したこと。 ControlNet の制御が壊れる条件を 3 つ切り出し、なぜ壊れるかまで実験で示した。推奨されている省メモリ設定が黙って出力を壊す件は、`get_attention_scores` が未初期化のまま渡す加算元が原因で、0 で埋めるだけで直る。条件地図の密度が下がると制御が消える件では、指定した輪郭との一致が 0.7329 から 0.2391 へ落ちて偶然の水準と区別できなくなる一方、生成画像自体は破綻しない。UNet へ注入される残差を直接測ると密度とともに縮んでおり、**制御が消えるのは残差が消えるからではなく、条件地図に由来する成分が縮むからである。** ここから密度を揃える前処理を作り、構造整合が改善してプロンプト追従は下がらないことを確かめた。三つ目の既定値が弱い件は、破綻ではなく二つの指標のあいだの妥協点として第 8 節で扱う。

1. 対象論文

Lymin Zhang, Anyi Rao, Maneesh Agrawala.

"Adding Conditional Control to Text-to-Image Diffusion Models."

Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV), 2023, pp. 3813–3824. DOI: 10.1109/ICCV51070.2023.00355 (本会議の full paper. 公開モデル `llyasviel/sd-controlnet-canny` が利用できる)

この論文が扱うのは、テキストだけでは指定できない条件を、学習済みの巨大な拡散モデルに後付けする問題である。「猫の絵」はプロンプトで頼めるが、「この輪郭に沿った猫」は言葉にならない。素朴には条件つきで再学習すればよいが、その規模の学習を条件ごとに繰り返す計算資源は個人にはない。論文が障害として挙げるのはむしろデータ量で、特定の条件では最大級のデータセットでも約 10 万枚と LAION-5B の 5 万分の 1 しかなく、その量で直接微調整すれば過学習と破滅的忘却を招くと述べている。

ControlNet の答えは、元のモデルには一切手を触れないというものである。重みを凍結したまま、そのエンコーダと中間ブロックの複製を作って条件を受け取らせ、複製の出力を zero convolution という 0 で初期化した層を通して元のモデルへ残差として加える。

同じ枠組みで条件の種類を差し替えられるのも特徴で、輪郭・深度・姿勢・領域分割がそれぞれ別の分岐として公開されている。本レポートが扱うのは輪郭 (Canny エッジ) による条件づけである。

2. 手法理解

ControlNet は学習済み拡散モデルの重みを凍結したまま、そのエンコーダ 12 ブロックと中間ブロックの複製 (trainable copy) を作り、両者を zero convolution で接続する。複製の出力は元の U-Net の 12 本のスキップ接続と中間ブロックに加算される。論文が与える式は

$$y_c = F(x; \Theta) + Z(F(x + Z(c; \Theta_{z1}); \Theta_c); \Theta_{z2})$$

である。ここで F は凍結された元のブロック、 c が条件 (本レポートでは Canny エッジ地図)、 Z が zero convolution である。 Z は重みとバイアスを 0 に初期化した 1×1 畳み込みなので、学習開始時点では両方の Z が 0 を返し、 $y_c = y$ すなわち元のモデルの出力と完全に一致する。論文がこの設計を採った理由として述べているのは、有害な雑音が微調整に影響しないようにパラメータを 0 から徐々に育てるためである。

残差の重み付けは適用条件を押さえておく必要がある。論文が解像度に応じた重み $w_i = 64/h_i$ (深いブロックほど強い) を導入するのは classifier-free guidance に関する箇所、条件画像を条件側の予測だけに加える場合の手当てである。常に全接続へ掛けるとは書かれていない。diffusers 0.31.0 もこれに対応し、`guess_mode` のときだけ `torch.logspace(-1, 0, ...)` の重みを掛け、既定では全ブロックに `controlnet_conditioning_scale` を一様に掛ける (models/controlnet.py L844-851)。移植の欠落ではない。

本レポートの実験は `guess_mode` を既定の無効のまま行っているため、残差には全ブロックに一律な係数が掛かっている (係数の値自体は第 4.3 節以降で振る)。第 5 節で段ごとの残差を比較するときには重みの補正を入れていないのはこのためである。推論時に露出しているつまみは `controlnet_conditioning_scale` (既定 1.0) で、上式の第 2 項に掛かる係数にあたる。

学習の設定で本レポートに直接関わるのは、**プロンプトの 50% を空文字に置き換えて学習している**点である。論文はその狙いを「条件画像の意味を直接認識する能力を高めるため」と述べている。つまりこのモデルは、テキストが無いときには条件地図そのものから意味を読み取るよう仕向けられている。

両方の Z が 0 を返すのは、論文が述べているとおり**学習開始の時点だけ**である。学習後の zero convolution は 0 で初期化された重みが育っているため、条件が空でも 0 を返すとは限らない。実際、学習済みの重みに全画素 0 の地図を与えても、UNet へ渡される残差は 0 にならない（第5節で測る）。加えて第2項は $Z(F(x + Z(c; \Theta_{z1}); \Theta_c); \Theta_{z2})$ であり、条件が消えても潜在変数 x が trainable copy を通る成分は残る。

したがって条件が情報を失ったときに起きるのは、第2項が 0 になることではない。**条件地図に由来する成分が失われ、残差の大きさが縮む**のである。学習が「条件地図から意味を読む」方向に偏っている以上、失われるのは輪郭の指定だけではない。第5節ではこの退化を残差の大きさとして測る。

論文自身のアブレーションが、この構造の二つの側面を示す。zero convolution を通常の畳み込みに置き換えると性能が軽量版と同程度まで落ち、論文はその理由を、微調整の途中で trainable copy の事前学習済みバックボーンが壊れるためだと説明する。一方その軽量版は、条件画像を解釈するには力不足でプロンプトが不十分な条件では失敗するとも述べている。制御の強さは追加ネットワークの表現力と、事前学習済みの表現を壊さずに残差を渡せるかの両方で決まり、zero convolution が担うのは後者である。

なお論文本文には Limitations に相当する節が置かれていない（arXiv 版・ICCV 版本体の双方で確認）。ControlNet 自身の限界として明示されているのは ICCV 版 supplementary の Figure 28 の一点で、条件画像の意味が誤って認識されると強いプロンプトでも悪影響を取り除きにくい、という内容である。同 supplementary が挙げる他の限界（複数制御・少データでの過学習）は、比較対象として置かれた「元の重みを直接微調整する構成」のものであって ControlNet の限界ではない。

3. 実行環境

計算機	Apple M1（8コアGPU）／RAM 16GB／macOS 14.4.1（23E224）
バックエンド	PyTorch 2.2.2 の MPS（CUDA なし）
ライブラリ	diffusers 0.31.0／transformers 4.44.2／OpenCV 4.11.0
モデル	Stable Diffusion v1.5 + sd-controlnet-canny。重みは計 1.43×10^9 パラメータで fp16 なら約 2.7GiB（配布されている fp32 の実体はディスク上 5.3GiB）
サンブラ	DPMSolver++ 2M（Karras）20ステップ、 512×512
生成速度	20ステップ・ 512×512 で 1枚あたり 76～184秒（ $n = 157$ 、中央値 106.5秒、平均 110.8秒）。第4.1節の切り分けと速度比較は DDIM 4ステップなので別条件である

4. Failure Case Analysis

4.1 推奨されている省メモリ設定が、沈黙したまま出力を壊す

最初の生成が一枚も成立しなかった。返ってきた画像は 512×512 の全画素が値 0、完全な黒である（平均 0.0、標準偏差 0.0）。例外も警告も出ず処理は正常終了するので、構造整合の指標だけを見ていれば edge F1 = 0.000 が並ぶだけで原因は分からない。

VAE で復号する前の潜在変数を取り出すと、その段階ですでに NaN が入っていた。fp32 の VAE で復号し直しても消えない。fp16 の VAE が飽和したという既知の話ではなく、UNet の順伝播そのものが壊れている。

原因は enable_attention_slicing() だった。発生条件を絞るため、精度 2 通り × slicing の設定 5 通りで潜在変数の NaN を直接調べた（DDIM 4 ステップ、同一シード）。

	slicing なし	auto	max	1	4
fp16	正常 (3.38)	NaN	NaN	NaN	NaN
fp32	正常 (4.46)	正常 (4.46)	正常 (4.46)	正常 (4.46)	正常 (4.46)

括弧内は潜在変数の絶対値の最大である。**fp16 と slicing の組み合わせでのみ発生し、分割の粒度に依存しない**。fp32 では slicing を有効にしても差は小数第 6 位にとどまる（4.457312～4.457321）。なお

auto はヘッド数（このモデルでは 8）の半分、max は 1 に解決されるので、実際に試した粒度は 1 と 4 の 2 通りである（fp32 の絶対値最大が auto と 4、max と 1 で一致することからも確かめられる）。

diffusers 0.31.0 では slicing の有無で attention の実装が入れ替わる。既定は F.scaled_dot_product_attention、slicing 有効時はスコア行列を構成する Attention.get_attention_scores を通り、後者は加算元を未初期化のまま確保する。

```
buf = torch.empty(...); scores = torch.baddbmm(buf, query, key.T, beta=0, alpha=scale)
```

beta = 0 なので加算元は参照されないはずで、BLAS の規約でもそう定められている。未初期化のまま渡す実装自体は規約に従っている。溢れの線は薄い。入力の実数値の最大は 8.6 で、最も次元の大きい段（160）で蓄積して係数を掛けても上限は 840 程度、fp16 の 65504 の 80 分の 1 にとどまる。

加算元を差し替えると挙動が変わる。**torch.empty を torch.zeros にするだけで NaN が消えた**。粒度 1・4・auto のいずれでも消え、潜在変数の絶対値の最大は 3.379~3.383 と slicing 無効時の 3.377 に近い値へ戻る。中身を直接測ると（exp10_buffer_content.py）、fp16 では 2944 回のうち 2847 回のバッファが非有限値を含み、fp32 は 0 回だった。規約が守られているかも調べた（exp11_beta_zero.py）。同じ入力で加算元を 0 と NaN に入れ替えると、**MPS では結果が一致せず、CPU では一致する**。fp16 に限らず fp32 でも一致しない。加算元が有限値なら MPS でも一致するので、壊れるのは非有限値のときだけである。MPS の baddbmm は beta = 0 でも $0 \times C$ を実際に計算しており、IEEE 754 の $0 \times \text{NaN} = \text{NaN}$ がそのまま出る。CPU は C を参照しない。

原因は、規約が保証するはずの「beta = 0 なら加算元は参照されない」が MPS で成り立たないことである。fp16 でだけ壊れて見えるのは演算の違いではなく、その時点のメモリに非有限値が残っていたかどうかの違いにすぎない。中身が何であれ結果に影響してはならない以上、その NaN の出所は欠陥の説明に要らない。

利用者の側で直すなら、加算元を 0 で初期化した実装に差し替えれば足りる（第 6 節に挙げる二つの改善のうちの一つ）。本番と同じ 20 ステップで確かめると、slicing を有効にしたままでも 5 枚すべてが正常に生成され、構造整合の平均は 0.7426 と slicing 無効時の 0.7427 に一致した。画素差も平均 0.18（0~255 階調）で、実質的に同じ画像が出る。fp32 に切り替えても回避できるが、fp16 の 23.6 秒に対し 39.9 秒と **1.7 倍遅くなる**（bench_dtype.py）。以降の実験は fp16 のまま slicing を無効にして行った。

問題は、この設定が推奨されている点にある。diffusers の MPS 最適化に関する公式文書は RAM 64GB 未満で enable_attention_slicing() の有効化を案内しており、本環境の 16GB は条件に当てはまる。案内どおりに従うと生成が全滅し、しかも失敗が静かなのでユーザーは自分の設定ミスを疑う。あわせて生成直後に NaN と単色を検出して例外を投げる検査を組み込んだ。静かに壊れる失敗は下流のすべての指標を無意味にする。

4.2 条件地図の密度が下がると制御が消える

Canny の閾値は公式実装の既定値（100, 200）に固定したまま、入力のコントラストだけを係数 α で落とした系列を作った。構図と内容は変えていないので、変化するのは条件地図の密度だけである。実画像 10 点（skimage.data 同梱）と、円と矩形の反復数だけを変えた合成画像 5 点に対し、 $\alpha \in \{0.15, 0.3, 0.5, 1.0\}$ を掛けた 60 条件を生成した。

構造整合は、生成画像から条件と同じ設定で Canny を再抽出し、入力の実数値条件地図との F1 で測る（2 画素の許容を入れた膨張マッチング。拡散生成では輪郭が数画素揺れるので、許容なしでは構造が保たれていても F1 がほぼ 0 になる）。

エッジ画素率と構造整合の間には強い順位相関があった（Spearman $\rho = +0.819$, $n = 60$ ）。この値はそのまま受け取れない。条件地図が空の 15 条件では F1 が定義上 0 に固定される（理由は次の表のあとで述べる）ので、それらを除いた 45 条件で取り直すと $\rho = +0.577$ ($p = 3.4 \times 10^{-5}$) になる。前者は定義上の 0 に押し上げられた値である。

条件地図のエッジ画素率	n	edge F1 平均	中央値	標準偏差
0（完全に消滅）	15	—	—	—
~0.005	6	0.109	0.012	0.229
~0.02	11	0.668	0.611	0.288
~0.05	14	0.721	0.748	0.145
0.05~	14	0.841	0.859	0.088

密度 0 の行を「—」としたのは、この指標がそこでは使えないからである。条件地図が空だと適合率は空虚に 0、再現率は 0/0 になり、生成画像が何であっても F1 が 0 を返す。Chamfer 距離も同じ理由で定義できない。「密度 0 で F1 が 0」は破綻の測定ではなく、指標の定義から自動的に出る値である。破綻の証拠にはできない。

代わりに 3 点を述べる。

第一に、60 条件のうち 15 条件でエッジ画素率がちょうど 0 になった。既定の閾値が入力のコントラストに対して高すぎるため、条件地図が真っ黒になる。Canny の閾値処理だけで決まるのでシードには依存しない。ただし 15 行のうち生成画像が異なるのは 8 枚だけで、残りは md5 まで一致する。条件地図が空になると元画像が違ってパイプラインへ渡る入力が一致するため、独立な観測は 8 件と数えるべきである。

第二に、条件が効いていないことを示すには別の測り方が要る。密度 0 の生成画像を、**本来指定したかった輪郭**（コントラストを落とさない条件地図）と照合すると F1 は平均 0.2391 ($n = 15$)。正しく条件づけた生成は同じ指標で 0.7329 である。つまり **0.7329 から 0.2391 へ落ちている**。この落差が主張の中心である。無関係な画像どうしを組み合わせた偶然一致の水準は 0.2461（総当たり 210 点なので独立ではない）で、密度 0 の 0.2391 がこれを上回るとは言えない（Mann-Whitney U 検定、 $p = 0.807$ ）。非有意は一致の証明ではないので、偶然の水準と同じだとまでは主張しない。重複を除いた 8 枚でも平均 0.2822 で結論は変わらない（ $p = 0.680$ ）。

壊れているのは制御であって、生成ではない。密度 0 の生成画像はエッジ画素率 0.0074~0.2539 を持つ普通の画像で、真っ黒でも崩れてもいない。chelsea は猫、rocket はロケットとプロンプトの語に対応する物が出るが、図1の camera では主題の人物まで落ちる。失われるのは輪郭の指定だけとは限らない。



図1 同一画像のコントラストだけを落としたときの条件地図（上段）と生成結果（下段）。プロンプトは全列で同一。 $\alpha = 0.15$ では条件地図が完全な黒になる（最大値 0）が、生成画像は破綻せず三脚とカメラの写真が出てくる。ただし人物が消え、構図は元画像と無関係になっている。密度/F1 は左から 0.0751/0.758、0.0253/0.628、0.0139/0.541、0/定義不能。

第三に、低密度域は成績が悪いだけでなく**ばらつきが大きい**。ただし帯ごとの標準偏差は主に入力画像の違いを反映するので、それだけでは生成の不安定さを示せない。そこで密度帯を代表する 5 条件を四分位で選び、シードだけを変えて 5 回ずつ生成した。条件内の標準偏差は平均 0.0644 で、条件平均の標準偏差 0.3543 の 5.5 分の 1 にとどまる。**密度の効果はシードの揺らぎでは説明できない**。一方、最も揺れる密度 0.0201 ではシードを変えるだけで edge F1 が 0.4965 から 0.8589 まで動く。壊れかけの領域では、制御が効くかどうか賭けになる。

4.3 推論時の既定値は構造整合に対して弱すぎる

当初は「最適な controlnet_conditioning_scale は入力の密度によって動く」という予想を立て、7 つの入力に対し $scale \in \{0.2, 0.4, 0.6, 0.8, 1.0, 1.3\}$ を振った。**この予想は外れた**。7 入力すべてで掃引範囲の上限である 1.3 が最良となり、密度との関係は現れなかった（最適値が全件同一のため順位相関は定義できない）。

外れ方から分かったこともある。**既定値 1.0 が最適だった入力は 7 件中 0 件**で、どの入力でも 1.0 から 1.3 へ上げると構造整合が改善した。改善幅は入力によって大きく異なり、密度の低い synth_sparse では

0.333 から 0.877 へ跳ねる一方、密度の高い synth_veryd では 0.957 から 0.978 にとどまる。

この結論は、構造整合という一つの指標だけを見た結果にすぎない。掃引範囲を狭く取ったために代償が見えていない。第 8 節では範囲を 3.0 まで延ばし、プロンプト追従 (CLIP 類似度) と併せて評価する。

5. 機構の直接証拠：注入される残差そのものを測る

第 4.2 節で示したのは入力と出力の対応にすぎない。密度が下がると成績が落ちるという相関は、条件が効かなくなったからだと、生成そのものが難しくなったからだと読める。第 2 節で立てた予測は前者、すなわち zero convolution を通って UNet に加わる残差が縮むというものだった。これは測れる。

ControlNet の順伝播を 1 回だけ実行し、UNet の各段へ渡される残差の二乗平均平方根を求めた。timestep は 500、潜在変数はシード 0 で固定した共通のものを使い、条件側の枝だけを見ている (classifier-free guidance の無条件側は測っていない)。これらを共有しているので、その影響を取り除いた比較になる。ただし系列間ではプロンプトも変わるため、条件地図の違いだけを取り出した比較が厳密に成り立つのは各画像のコントラスト系列の内部である。拡散の全過程を回さないで 1 条件あたり 1 秒未満で済み、コントラスト係数を 9 段階に振れた。

密度と残差の間には強い順位相関があった (Spearman $\rho = +0.875$ 、順伝播 135 回。画像ごとに値が重複する行を除いた 83 件でも +0.874。この +0.875 には密度 0 の 45 回が寄与しており、密度が正の 90 回に限ると +0.669、その重複を除いた 68 件では +0.784 である)。

条件地図のエッジ画素率	順伝播の回数	総残差 RMS	mid ブロックの RMS
0 (完全に消滅)	45 (異なる値は 11。違いはプロンプト)	0.3846	0.9333
~0.005	11	0.5161	1.2990
~0.02	28	0.8027	1.8046
~0.05	27	0.9785	2.0590
0.05~	24	1.1114	2.0724

回数の列は独立な観測数ではない。順伝播は決定論的なので、同一の条件地図を与えた行はビット単位で一致する。密度 0 の 45 行が取る値の範囲は 0.3608~0.4090 (標準偏差 0.0150) である。

密度 0 の条件地図は全画素 0 の配列なので、参照として用意した空白の地図と**同一である**。両者の残差が近くなるのは当然で、密度を動かしていない比較は密度について何も語らない。この比較は主張に使わない。

機構として意味を持つ領域は、密度が正の側である。そこでの減衰を見る。総残差は 1.1114 から 0.5161 へ単調に下がり、密度 0 まで含めれば 0.3846 に達する。画像ごとにコントラスト系列内で順位相関を取ると、15 画像の中央値が +1.000、11 画像が +0.98 以上、残る 4 画像も +0.03~+0.54 で全画像が正だった。画像の内容を固定して密度だけを動かしても残差が縮むので、画像間の差では説明できない。

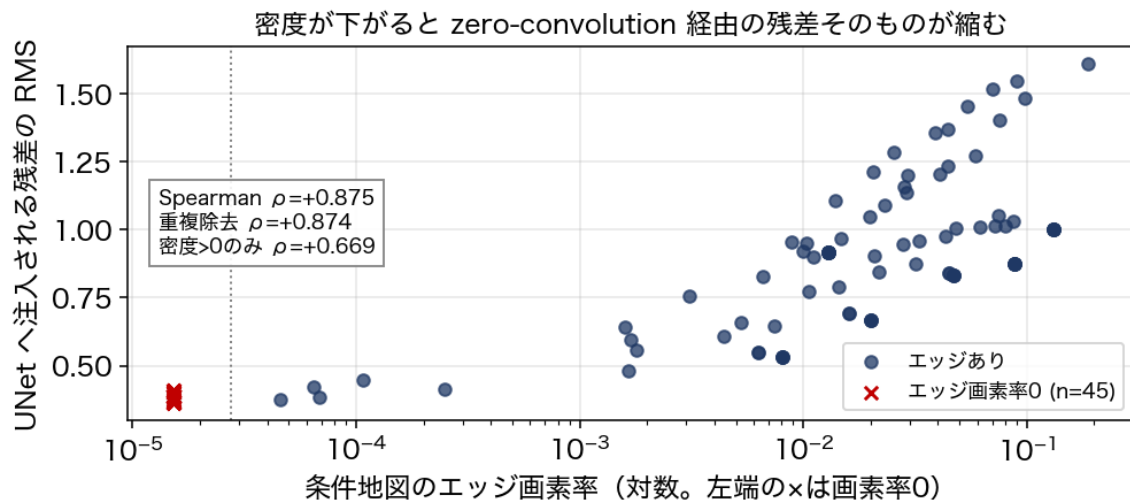


図2 条件地図のエッジ画素率と、UNet へ注入される残差の大きさ。左端の × は画素率が 0 の 45 回の順伝播で、これらは全画素 0 の同一の地図によるものなので値は 11 通りしかない (違いはプロンプトによる)。timestep と潜在変数は全条件で固定してあるが、系列ごとにプロンプトも変わるため、密度だけを動かした比較が厳密に成り立つのは各画像の系列内である。点線より左は対数軸に載らない画素率 0 を別枠で置いたもので、横位置そのものに意味はない。本文が機構の根拠に据えているのは密度が正の領域の減衰 ($\rho = +0.669$) である。

第2節で述べたように、条件が失われても第2項は0にはならない。残ったものが何かを確かめるため、残差を空間方向に分解した (probe_spatial.py)。チャンネルごとの空間平均を引いた成分、すなわち空間的に変化している分の割合を測る。第4.2節で条件が消えた camera ($\alpha = 0.15$) では、mid の RMS 0.9889 のうち 0.6675 (67.5%) が空間成分で、12本のスキップ接続のうち7本目でも 48.4% を占める。**残差は一樣ではない**。原画像では mid の RMS が 3.2323・空間成分 84.5% なので、条件が消えると残差は 3.3 分の 1 に縮み、空間的に変化している割合も下がる。

条件が失われたときに起きるのは、残差が消えることでも一樣な項になることでもなく、**条件地図に由来する成分が縮むこと**である。本節で測ったのは残差だけである。出力側で何が起きるかは第4.2節で別に測った。

6. 改善と、その前後比較

本レポートの改善は二つある。一つは第4.1節で述べた未初期化バッファの一行修正で、推論の前段が静かに壊れる問題を直す。もう一つが本節で扱う、制御そのものの品質の改善である。第4.2節と第5節から、条件が効かなくなる原因は閾値が固定されていることに帰着する。入力のコントラストが変われば同じ閾値でもエッジ画素率が桁で動き、極端な場合は0になる。ならば閾値ではなく**密度を揃えればよい**。

実装は二つの部品からなる。一つは目標エッジ画素率 (0.045) へ近づくよう Canny の高閾値を 12 回二分探索し、その中で目標に最も近い密度の地図を採る前処理で、探索の前に CLAHE で局所コントラストを補正する。もう一つは、得られた密度に応じて controlnet_conditioning_scale を線形に調整する規則である。どちらも拡散を回す前の処理なので、**生成 1 枚あたりの計算コストは増えない**。

評価は同一の画像・プロンプト・シードで改善前と改善後を対にして行った。15 画像 × コントラスト 2 水準 × シード 3 個で 90 対である。ここで検定の単位に注意が要る。1 枚から 6 行出るので、90 行をそのまま対応のある検定にかけると標本数が水増しされる。シードは画像内で平均し、画像を単位とした。さらに**コントラストの水準を混ぜてはいけない**。混ぜると符号が打ち消し合う。

参照地図の取り方にも落とし穴がある。前後をそれぞれが与えられた地図で測ると、F1 が参照地図の密度に依存するぶんが改善幅に混入する。そこで両者を**同じ参照**、第4.2節と同じ「本来指定しなかった輪郭」で測り直した値も併記する。

コントラスト	n (画像)	改善した画像	Δ 中央値	Wilcoxon p
$\alpha = 0.3$ (低コントラスト)	15	12 / 15	+0.1194	0.041
$\alpha = 1.0$ (原画像)	15	5 / 15	-0.0784	0.188

低コントラスト側では改善し、原画像側では逆向きに振れている。二層を平均すると相殺されて「差がない」になるが、それは実態を隠す。共通参照で測り直すと $\alpha = 0.3$ は Δ 中央値 +0.0665・ $p = 0.055$ 、 $\alpha = 1.0$ は -0.0560 ・ $p = 0.107$ で、**改善の向きは保たれるが大きさは半分に落ちる**。

低コントラスト側で効くのは筋が通っている。改善前の平均密度 0.0270 が 0.0354 へ上がり目標に近づく。条件地図が空になっていた 2 画像では密度が 0 から平均 0.0346 へ回復した。この 2 画像は改善前の F1 が定義上 0 なので、数値は「指標が使える状態に戻せたか」を示すもので、改善幅としては読めない。

原画像側が悪化の向きに振れる理由は、二分探索が目標に届いていないことにある。 $\alpha = 1.0$ の 15 画像のうち適応後の密度が目標 0.045 の $\pm 10\%$ に収まったのは 5 画像で、平均は 0.0647 から 0.0630 へわずかしき動かない。高閾値の探索範囲を 20~400 に切っているため実画像では上端でも密度が落ちきらず、合成画像は密度が閾値に対して階段状に変わるので動かない。scale を決める規則は**適応後**の密度を読むので、高いまま残った密度から scale が平均 0.854 まで下がり、構造整合が下がる。ただしこの層の悪化は $p = 0.188$ で有意ではない。

この $\alpha = 0.3$ の検定には、指標が定義から壊れている画像の寄与も混じる。二分探索が密度を 0 まで落とした synth_veryd を除くと 14 画像・ $p = 0.0040$ と強くなり、改善幅として読めない cell・chelsea を除くと 13 画像・ $p = 0.110$ で有意でなくなる。 **$\alpha = 0.3$ の有意性は標本数の小ささに左右される程度のものである**。(synth_veryd は元の密度が目標を上回るので、次に述べる適用条件では対象から外れる。)

ここまで構造整合だけを見てきたが、それでは第4.3節と同じ誤りを繰り返す。生成済み画像に CLIP を後付けで測ると、プロンプト追従は $\alpha = 0.3$ で 0.2911 → 0.2967、 $\alpha = 1.0$ で 0.2931 → 0.2984 とどちらも有意に動かない ($p = 0.52$ と 0.76)。scale を大きく振れば追従は下がるが (第8節)、この改善が動かす幅では代償が出ていない。

推奨する形そのものも評価した。**密度が目標を下回るときにだけ適用する**という一行の判定を足すと、 $\alpha = 0.3$ では 13 / 15 画像が対象になり共通参照での p が 0.055 から 0.0019 へ下がる。 $\alpha = 1.0$ では 6 / 15 にしか適用されず、悪化していた Δ 中央値 -0.0784 が 0.0000 になる。**害を止めたうえで、効く場面を残せている**。ここまでは規則を導いたのと同じ 15 画像での評価である。そこで、改善も適用条件も作るのに使っていない skimage.data の 7 画像を別に用意し、規則を変えずに当てた（密度は $\alpha=1.0$ で 0.0005～0.3130 と、開発側の 0.0016～0.1884 より広い）。

低コントラスト側は 7 画像中 6 画像で改善し、共通参照での Δ 中央値は $+0.0788$ だった。ただし 7 画像では有意水準に届かない ($p = 0.078$)。原画像側は無条件に適用すると 6 画像で悪化し、 Δ 中央値 $-0.0799 \cdot p = 0.031$ と有意に害が出る。適用条件を入れると対象が 3 画像に絞られ、 Δ 中央値は 0.0000 ($p = 0.500$) へ戻る。**改善の向きは開発に使っていない入力でも保たれ、適用条件は効かない場面での害を止めている**。効果量そのものは、この標本数では有意差として示せない。

7. Limitations

本レポートの結論には以下の制約がある。

指標の設計に依存している。構造整合は再抽出した Canny との F1 で測るが、2 画素の許容幅・再抽出時の閾値・膨張マッチングに依存する。条件地図が空だと定義から 0 を返すので、破綻そのものをこの指標では測れない。代替の偶然一致水準との比較も「本来指定したかった輪郭」を人為的に選んでいる。

標本数が小さい。第 6 節の改善前後比較は片側 15 画像、シードは 3 個で、有意性が個々の画像の扱いで動く程度の検出力しかない。開発に使っていない 7 画像でも確かめたが、そちらはさらに小さく、低コントラスト側の改善は有意水準に届かない。第 4.2 節のシード分散も 5 条件 $\times$ 5 シードにとどまる。

輪郭条件しか扱っていない。深度・姿勢・領域分割でも同じことが起きるかは調べていない。密度に相当する量を条件の種類ごとにどう定義するかという問題も残る。

環境とモデルに固有の可能性がある。第 4.1 節の欠陥は PyTorch 2.2.2 の MPS で確かめた。CPU では規約が守られることも測ったが、CUDA など他のバックエンドや他の版は扱っていない。生成側も Stable Diffusion v1.5 系に限られ、SDXL 以降での挙動は検証していない。ただし推奨設定に従うと例外を出さずに壊れる構図そのものは、環境が違って起こりうる。

論文自身の主張を検証したものではない。扱ったのは推論時の使い方であり、論文が報告する学習の性質や他手法との比較は追試していない。ここで挙げた限界は、第 2 節で触れた Figure 28 とは別に本実験から見出したものである。

8. 考察

第 4.3 節では scale を 1.3 まで振って、上げるほど構造整合が良くなるという結果を得た。そこで掃引を 3.0 まで延ばし、同時に CLIP 類似度でプロンプト追従を測った。

scale (10 段階のうち 7 段階を抜粋)	0.2	0.6	1.0	1.6	2.0	2.5	3.0
edge F1	0.251	0.589	0.717	0.909	0.925	0.920	0.919
CLIP 類似度	0.304	0.302	0.291	0.273	0.271	0.262	0.258

構造整合は 2.0 で頭打ちになるが、プロンプト追従はそこを越えても下がり続ける。この掃引も 7 画像から 10 段階ずつなので、全 70 点を独立標本にはできない。画像ごとに系列内で順位相関を求めると、構造整合は中央値 $\rho = +0.915$ で**7 画像すべてが正**、プロンプト追従は中央値 $\rho = -0.673$ で**7 画像中 5 画像が負**だった。

向きが逆であることは表からも画像内の相関からも一貫している。ただし**プロンプト追従の低下は統計的に言い切れない**。7 画像中 2 画像で相関がほぼ 0 か弱い正（最大 $+0.067$ ）で、Wilcoxon の符号順位検定は有意水準に届かない ($p = 0.078$)。標本が 7 画像しかないのである。**第 4.3 節で得た「上げるほど良い」が片方の指標だけを見ていたために生じた見え方だったことは、平均値の推移からは言える**。

両方を正規化して和を最大化する scale を入力ごとに求めると 0.6 から 3.0 まで散る。構造整合だけで選ぶと全入力 が 1.6～3.0 に集まるのに、両立を求めると入力ごとに異なる。ただし第 4.3 節の予想は「最適な scale は入力の**密度**によって動く」であり、密度との対応は両指標で見ても現れない。予想は外れたままで、成立したのは「入力によって動く」という弱めた形までである。**何を測るかによって予想の当たり外れが変わったことになる**。

ここに本レポートを通じた見方が現れている。条件の強さは、増やすほど良い資源ではない。第 4.2 節と第 5 節で見たのは資源が足りない側の破綻、本節で見たのは資源が過剰な側の代償である。密度が低い入力でも scale を上げれば構造整合は改善するが (synth_sparse は 0.333→0.877)、条件地図が空になった入力では増幅しても指定は戻らない。両者を同時に振ってはいないので交互作用は測れていない。**密度と scale は、条件の実効的な強さという一つの量に対する別々の支払い方である。**最適値は入力の中ではなく、入力の構造と何を達成したいかの組み合わせの中にある。

筆者は大学院で、資源を増やすほど成績が上がるとは限らず、その価値がタスクの構造との整合で決まることを扱っている。本レポートの条件の強さも、その意味での資源の一例である。とはいえ単一の手法・単一の条件タイプにすぎない。むしろ有用なのは逆向きの示唆で、**手法の既定値は目的が一つに定まっているときの値であり、測る指標を変えれば既定値の妥当性も変わる**という点である。

controlnet_conditioning_scale = 1.0 は構造整合では明らかに弱い、プロンプト追従を含めれば妥当な妥協点になっている。既定値が「弱すぎる」と読めたのは、片方しか測っていなかったからである。

9. 生成AI利用

使用したもの: Claude (Anthropic)。対話と自律実行の両方の形で用いた。

用途: 実験コードの作成、統計処理と作図、本レポートの文章の下書き、完成前の批判的レビュー。生成AIが出すのは下書きと実装であり、**何を主張し何を撤回するか、どこまで検証を求めるかは筆者が決めている。**

生成AI が書いたものをそのまま採らないという方針を先に決め、二つの仕組みを用意した。

stats_report.py は本文が挙げる統計量を節ごとに再現する。audit_numbers.py は本文中の数値を抜き出し、その数字列が stats_report.py の出力か results/*.csv に現れるかを照合して、現れないものを「出所不明」として挙げる。数字列の一致を見るだけで統計量の意味までは見ないが、出所の無い数値が紛れ込むことは防げる。残るのは書誌情報とバージョン、実験設定として登録した定数だけである。

実際に差し戻した箇所を挙げる。いずれも生成AIの出力を疑って一次資料に当たった結果である。会議名を「CVPR 2023」と提示されたので、Crossref で DOI を照合して ICCV 2023 に訂正した。論文の扱いでも「解像度依存の重みが diffusers の既定では効いていない」と食い違いを指摘していたが、読み直すとその重みは classifier-free guidance の特定の場合の手当てで、guess_mode が正しく対応していた。実装の不備ではないので撤回した。

主張の根拠に関わる訂正が二件ある。「条件地図が空のとき edge F1 が例外なく 0 になる」と「空の条件地図の残差が参照値とほぼ一致する」を、いずれも破綻の証拠として書いていた。どちらも指標や比較の定義から出る値で、測定ではない（理由は第 4.2 節と第 5 節）。撤回し、偶然一致の水準との比較と、密度が正の領域での減衰に置き換えた。

検定の設計も直した。標本数の水増しとコントラストの層別、改善前後で参照地図が違う点である（いずれも第 6 節）。

生成AI がレビュー過程で測定値を提示した箇所もある。**他人が出した数値を自分の主張の根拠にはしない**という方針を採り、同じ測定を実装し直して再現を確認してから載せた。再現できなかった数値は外した。

参考文献

- [1] Lvmin Zhang, Anyi Rao, Maneesh Agrawala. "Adding Conditional Control to Text-to-Image Diffusion Models." Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV), 2023, pp. 3813–3824. DOI: 10.1109/ICCV51070.2023.00355 (Crossref・DBLP・Semantic Scholar で照合。supplementary は CVF 公開版)
- [2] Robin Rombach, Andreas Blattmann, Dominik Lorenz, Patrick Esser, Björn Ommer. "High-Resolution Image Synthesis with Latent Diffusion Models." CVPR, 2022, pp. 10674–10685. DOI: 10.1109/CVPR52688.2022.01042 (Crossref で照合)
- [3] Hugging Face. "Metal Performance Shaders (MPS)." diffusers documentation. <https://huggingface.co/docs/diffusers/en/optimization/mps>
- [4] diffusers v0.31.0 実装 (models/controlnet.py, models/attention_processor.py)
- [5] Stéfan van der Walt et al. "scikit-image: image processing in Python." PeerJ 2:e453, 2014. DOI: 10.7717/peerj.453 (帰属は LICENSE_DATA.md)